Problemas: Tema 04

En todos los problemas se deberá analizar el enunciado, identificar las entidades que intervienen y diseñar las clases necesarias para representarlas. Se implementarán las clases en Java, separando su definición de la clase conductora que contendrá el método `main` y permitirá probar su funcionamiento.

Las operaciones necesarias para trabajar con los objetos se ofrecerán mediante métodos públicos. Se deberá prestar atención a la inicialización de los objetos, a la coherencia de su estado y a la diferencia entre los métodos observadores y los transformadores.

En los problemas que impliquen varias instancias, se deberán crear distintos objetos y comprobar que cada uno mantiene su propio estado. Cuando se trabaje con referencias compartidas, se deberá analizar qué variables permiten acceder a cada objeto y qué consecuencias tienen las modificaciones realizadas sobre una instancia.

1. Puntos en el plano

Diseña e implementa una clase `Punto` que represente un punto en un plano cartesiano. Cada punto estará definido por sus coordenadas X e Y, ambas de tipo `double`.

La clase deberá disponer de un constructor que permita crear un punto en el origen de coordenadas y otro que permita establecer sus coordenadas iniciales. También deberá proporcionar las siguientes operaciones:

- Consultar individualmente cada una de las coordenadas.
- Desplazar el punto una determinada distancia horizontal y vertical.
- Calcular la distancia entre el punto actual y otro punto recibido como parámetro.
- Determinar si el punto se encuentra en el origen de coordenadas.

Implementa una clase conductora que cree varios puntos, realice diferentes desplazamientos y calcule las distancias entre ellos. Comprueba que modificar uno de los objetos no altera el estado de los demás.

2. Referencias y alias de objetos

Utiliza la clase `Punto` del problema anterior para analizar el comportamiento de las variables de referencia y el paso de parámetros en Java.

Considera el siguiente programa:

```
class PruebaReferencias {

    static void modificar(Punto punto) {
        punto.desplazar(1, 2);
        punto = new Punto(0, 0);
    }
```

```
    public static void main(String[] args) {
        Punto origen = new Punto(2, 3);
        Punto alias = origen;
        Punto copia = new Punto(2, 3);

        modificar(alias);

        System.out.println(origen.getX() + ", " + origen.getY());
        System.out.println(alias.getX() + ", " + alias.getY());
        System.out.println(copia.getX() + ", " + copia.getY());

        System.out.println(origen == alias);
        System.out.println(origen == copia);
    }
}
```

Realiza una traza del programa y responde a las siguientes cuestiones:

- ¿Cuántos objetos `Punto` se crean durante la ejecución?
- ¿Qué variables contienen referencias al mismo objeto?
- ¿Qué valores se muestran por pantalla?
- ¿Por qué la asignación de una nueva instancia a la variable `punto` no modifica las referencias almacenadas en `origen` y `alias`?
- ¿Qué diferencia existe entre modificar el estado de un objeto mediante un parámetro y modificar el valor de la referencia recibida?
- Representa la evolución de las variables y los objetos en las estructuras Stack y Heap durante la ejecución.

Finalmente, modifica el programa para que `origen` reciba el valor `null`. Comprueba si el objeto al que hacía referencia sigue siendo accesible mediante alguna de las otras variables. Explica en qué momento una instancia puede quedar disponible para el recolector de basura.

3. Rectángulos y sobrecarga de constructores

Diseña e implementa una clase `Rectangulo` que represente un rectángulo mediante sus dimensiones: base y altura.

La clase deberá disponer de tres constructores:

- Un constructor sin parámetros que cree un rectángulo de dimensiones 1×1.
- Un constructor con un único parámetro que cree un cuadrado.
- Un constructor con dos parámetros que permita establecer la base y la altura.

Utiliza el encadenamiento de constructores mediante la palabra reservada `this` para evitar repetir las operaciones de inicialización que sean comunes.

La clase deberá ofrecer métodos para calcular el área y el perímetro, consultar las dimensiones, modificar las dimensiones y comprobar si el rectángulo es un cuadrado.

Implementa una clase conductora que cree rectángulos utilizando las tres alternativas de inicialización. Comprueba que los constructores establecen correctamente los valores y que las operaciones posteriores modifican únicamente el objeto correspondiente.

4. Gestión de cuentas bancarias

Una entidad bancaria necesita una aplicación sencilla para gestionar cuentas de sus clientes. De cada cuenta se desea almacenar el nombre del titular y el saldo disponible.

Diseña e implementa una clase `CuentaBancaria` que permita:

- Crear una cuenta indicando el titular y el saldo inicial.
- Consultar el titular y el saldo.
- Ingresar una cantidad en la cuenta.
- Retirar una cantidad, siempre que exista saldo suficiente.
- Transferir una cantidad desde la cuenta actual a otra cuenta recibida como parámetro.

Las operaciones de ingreso y retirada deberán impedir cantidades negativas o iguales a cero. La retirada y la transferencia no podrán dejar la cuenta de origen con saldo negativo. Los métodos que realicen una operación deberán indicar si esta se ha podido completar.

Implementa una clase conductora que cree varias cuentas y realice operaciones de ingreso, retirada y transferencia. Comprueba los saldos después de cada operación, incluidos los casos en los que una operación no pueda realizarse.

5. Operaciones con fracciones

Diseña e implementa una clase `Fraccion` que permita representar números racionales mediante un numerador y un denominador, ambos de tipo entero.

La clase deberá permitir crear fracciones indicando ambos valores y comprobar que el denominador no sea cero. También deberá proporcionar operaciones para:

- Consultar el numerador y el denominador.
- Simplificar la fracción.
- Obtener la fracción inversa.
- Sumar dos fracciones.
- Multiplicar dos fracciones.

Las operaciones de suma y multiplicación recibirán otra fracción como parámetro y devolverán una nueva instancia con el resultado correspondiente. No deberán modificar las fracciones originales.

Para simplificar una fracción se utilizará el máximo común divisor del numerador y el denominador. Se deberá tener en cuenta el signo de la fracción para mantener una representación coherente.

Implementa una clase conductora que cree varias fracciones y realice distintas operaciones con ellas. Comprueba que los resultados sean correctos y que las fracciones originales conserven sus valores después de las operaciones.

6. Simulación del comportamiento de una ameba

Las amebas son seres unicelulares cuyo peso puede variar al alimentarse. Diseña e implementa una clase `Ameba` que represente una ameba mediante un atributo `peso`, expresado en microgramos y de tipo entero.

Cuando se crea una ameba, su peso inicial será de 3 microgramos. La clase deberá proporcionar métodos para consultar el peso y alimentarse de dos formas diferentes.

En primer lugar, una ameba puede fagocitar una partícula de alimento. Durante este proceso consume un microgramo de su propio peso y el resto de la partícula se incorpora a su organismo. Por ejemplo, si una ameba de 3 microgramos ingiere una partícula de 6 microgramos, su peso final será de 8 microgramos.

En segundo lugar, una ameba puede alimentarse de otra ameba. En este caso, la ameba que se alimenta deberá tener un peso superior al de la ameba que intenta ingerir. Si puede hacerlo, consumirá un microgramo durante el proceso de fagocitación y el resto del peso de la otra ameba se incorporará al suyo. La ameba ingerida no se elimina del programa, pero su peso pasará a ser cero y ya no podrá alimentarse.

Por ejemplo, si una ameba de 7 microgramos se come a otra de 4 microgramos, la primera terminará pesando 10 microgramos y la segunda tendrá un peso de cero. Sin embargo, una ameba de 4 microgramos no podrá comerse a otra de 7 microgramos.

La clase deberá impedir que una ameba pueda alimentarse de sí misma o de otra cuyo peso sea cero.

Implementa una clase conductora que cree varias amebas, realice diferentes operaciones de alimentación y muestre el peso de cada una después de las operaciones. Comprueba tanto las situaciones en las que la alimentación es posible como aquellas en las que debe rechazarse.

7. Contadores y miembros de clase

Diseña e implementa una clase `Contador` que permita representar contadores independientes. Cada objeto dispondrá de un atributo que almacene su valor actual y ofrecerá métodos para consultar el valor, incrementarlo, decrementarlo y reiniciarlo.

Todos los objetos de la clase deberán comenzar con un valor inicial de cero. Para evitar repetir este valor en distintos lugares de la implementación, se declarará una constante de clase denominada `VALOR_INICIAL`.

Además, la clase deberá mantener un contador estático que registre el número total de instancias de `Contador` que se han creado desde que se carga la clase. Cada vez que se cree una nueva instancia, este contador deberá incrementarse.

Se deberá proporcionar un método estático que permita consultar el número de instancias creadas sin necesidad de crear un objeto adicional.

Implementa una clase conductora que cree varios contadores, modifique sus valores de forma independiente y consulte el número total de instancias creadas después de cada nueva creación.

Comprueba que los atributos de instancia mantienen valores independientes, mientras que el atributo estático almacena un valor compartido por todos los objetos de la clase. Explica por qué el método que consulta el número total de instancias puede invocarse utilizando directamente el nombre de la clase.

8. Control de temperatura mediante un resguardo

Un sistema de calefacción necesita conocer la temperatura de una estancia para decidir si debe activar o no la calefacción. El sistema real dispone de un sensor físico, pero todavía no está disponible para realizar las pruebas del programa.

Diseña una clase `ControlCalefaccion` que reciba en su constructor una referencia a un objeto encargado de proporcionar la temperatura actual. La clase deberá disponer de un método que compruebe si la temperatura está por debajo de los 18 grados y devuelva un valor lógico que indique si es necesario activar la calefacción.

Para probar el sistema sin disponer del sensor real, implementa una clase `SensorTemperatura` que actúe como resguardo (*stub*). Esta clase almacenará una temperatura establecida en su constructor y devolverá siempre ese valor cuando se solicite una lectura.

Finalmente, implementa una clase conductora que cree distintos sensores de prueba con temperaturas de 15, 18 y 22 grados. Utiliza cada uno de ellos para crear un objeto `ControlCalefaccion` y comprueba que la decisión del sistema es correcta en todos los casos.

Explica qué papel desempeña la clase conductora y por qué `SensorTemperatura` puede considerarse un resguardo de la dependencia que utilizaría el sistema real.

Tema 4